1.Implementation of the symbol table.

```
#include <stdio.h>
struct Symbol {
   char name[20], type[20], value[20];
};
int main() {
   struct Symbol s[100];
   int n, i;
   printf("Enter number of symbols: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++) {
      printf("\nEnter symbol name: ");
      scanf("%s", s[i].name);
      printf("Enter data type: ");
      scanf("%s", s[i].type);
      printf("Enter value: ");
      scanf("%s", s[i].value);
   }
   printf("\n------ SYMBOL TABLE ------\n");
   printf("Symbol\tType\tValue\n");
   for(i = 0; i < n; i++)
      printf("%s\t%s\t%s\n", s[i].name, s[i].type, s[i].value);
   return 0;
}
```

2. Develop a lexical analyzer to recognize a few patterns in C (ex. Identifiers, constants, comments, operators etc.)

```
#include <stdio.h>
#include <ctype.h>
int main() {
   char s[100];
   int i = 0;
```

```
    printf("Enter a string: ");
    fgets(s, 100, stdin);
    while(s[i] != '\0') {
        if(isalpha(s[i]) || s[i] == '_')
            printf("%c is an Identifier\n", s[i]);
        else if(isdigit(s[i])) {
            printf("%c", s[i]);
            while(isdigit(s[i+1]))
                printf("%c", s[++i]);
            printf(" is a Constant\n");
        }
        else if(s[i] == '+' || s[i] == '-' || s[i] == '*' ||
                s[i] == '/' || s[i] == '=')
            printf("%c is an Operator\n", s[i]);
        else if(!isspace(s[i]))
            printf("%c is a Special Symbol\n", s[i]);
        i++;
    }
    return 0;
}
```

3. Implementation of lexical analyzer using lex tool.

```
%{
#include <stdio.h>
%}
%%
[a-zA-Z_][a-zA-Z0-9_]*  { printf("%s : Identifier\n", yytext); }
[0-9]+                  { printf("%s : Constant\n", yytext); }
"+"|"-"|"*"|"/"|"="     { printf("%s : Operator\n", yytext); }
"//".*                  { printf("%s : Comment\n", yytext); }
[ \t\n]+                ;
```

```
.                   { printf("%s : Special Character\n", yytext); }
%%
int main()
{
   printf("Enter input:\n");
   yylex();
   return 0;
}
int yywrap()
{
   return 1;
}
```

4. Write a lex program to count the number of words and number of lines in a given file or program.

```
%{
#include <stdio.h>
%}
%%
[a-zA-Z_][a-zA-Z0-9_]*  { printf("%s : Identifier\n", yytext); }
[0-9]+              { printf("%s : Constant\n", yytext); }
"+"|"-"|"*"|"/"|"="    { printf("%s : Operator\n", yytext); }
"//".*              { printf("%s : Comment\n", yytext); }
[ \t\n]+             ;
.                   { printf("%s : Special Character\n", yytext); }
%%
int main()
{
   printf("Enter input:\n");
   yylex();
   return 0;
}
```

```
int yywrap()
{
   return 1;
}
```

5. Generate YACC specification for a few syntactic categories.

a. Program to recognize a valid arithmetic expression that uses operators +,-, *, and /.

lexer.l — LEX

```
%{
#include "y.tab.h"
%}
%%
[0-9]+  { return NUMBER; }
[ \t]   ;
\n      { return 0; }
.       { return yytext[0]; }
%%
int yywrap() {
   return 1;
}
```

lexer.y — YACC

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
expr : expr '+' expr
     | expr '-' expr
```

```
    | expr '*' expr
    | expr '/' expr
    | '(' expr ')'
    | NUMBER
    ;
%%
void yyerror(const char *s) {
   printf("Invalid Expression\n");
}
int main() {
   printf("Enter expression: ");
   if (yyparse() == 0)
      printf("Valid Expression\n");
   return 0;
}
```

5b. b. Program to recognize a valid variable that starts with a letter followed by any number of letters or digits.

lexer.y — YACC

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token LETTER DIGIT
%%
variable : LETTER
        | LETTER chars
        ;
chars : LETTER chars
     | DIGIT chars
     | LETTER
```

```
    | DIGIT
    ;
%%
void yyerror(const char *s) {
   printf("Invalid Variable\n");
}
int main() {
   printf("Enter a variable: ");
   if (yyparse() == 0)
      printf("Valid Variable\n");
   return 0;
}
```

lexer.l — LEX

```
%{
#include "y.tab.h"
%}
%%
[a-zA-Z]  { return LETTER; }
[0-9]     { return DIGIT; }
[ \t]     ;
\n        { return 0; }
.         { return yytext[0]; }
%%
int yywrap() {
   return 1;
}
```

5c. c. Implementation of calculator using Lex and YACC.

lexer.l — LEX

```
%{
#include "y.tab.h"
#include <stdlib.h>
```

```
%}
%%
[0-9]+  { yylval=atoi(yytext); return NUMBER; }
[ \t]   ;
\n      { return '\n'; }
.       { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

lexer.y — YACC

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
input : expr '\n' { printf("Result = %d\n",$1); }
    ;
expr : expr '+' expr { $$=$1+$3; }
    | expr '-' expr { $$=$1-$3; }
    | expr '*' expr { $$=$1*$3; }
    | expr '/' expr { $$=$1/$3; }
    | '(' expr ')'  { $$=$2; }
    | NUMBER        { $$=$1; }
    ;
%%
void yyerror(const char * *s) {
   printf("Invalid Expression\n");
}
```

```
int main() {
   printf("Enter expression: ");
   yyparse();
   return 0;
}
```

6. Convert the BNF rules into YACC form and write code to generate an abstract syntax tree.

Lexer Program – lexer.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+      { yylval.num = atoi(yytext); return NUMBER; }
[ \t\n]     ;
.           { return yytext[0]; }
%%
int yywrap()
{
   return 1;
}
```

2. YACC Program – parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
struct node
{
   char value[20];
   struct node *left, *right;
};
struct node* create(char *v, struct node *l, struct node *r)
{
```

```
    struct node *n = malloc(sizeof(struct node));
    sprintf(n->value, "%s", v);
    n->left = l;
    n->right = r;
    return n;
}
struct node *root;
void display(struct node *r, int level)
{
    int i;
    if(r == NULL)
        return;
    display(r->right, level + 1);
    for(i = 0; i < level; i++)
        printf("    ");
    printf("%s\n", r->value);
    display(r->left, level + 1);
}
int yylex();
void yyerror(char *s);
%}
%union
{
    int num;
    struct node *ptr;
}
%token <num> NUMBER
%type <ptr> input expr term factor
%left '+' '-'
%left '*' '/'
%start input
```

```
%%
input : expr
    {
       root = $1;
    }
    ;
expr  : expr '+' term
    {
       $$ = create("+", $1, $3);
    }
    | expr '-' term
    {
       $$ = create("-", $1, $3);
    }
    | term
    {
       $$ = $1;
    }
    ;
term  : term '*' factor
    {
       $$ = create("*", $1, $3);
    }
    | term '/' factor
    {
       $$ = create("/", $1, $3);
    }
    | factor
    {
       $$ = $1;
    }
```

```
    ;
factor : '(' expr ')'
     {
        $$ = $2;
     }
     | NUMBER
     {
        char str[20];
        sprintf(str, "%d", $1);
        $$ = create(str, NULL, NULL);
     }
     ;
%%
void yyerror(char *s)
{
   printf("Invalid expression\n");
}
int main()
{
   printf("Enter arithmetic expression: ");
   if(yyparse() == 0)
   {
      printf("\nAbstract Syntax Tree:\n");
      display(root, 0);
   }
   return 0;
}
```

7. Write a YACC program to implement a top-down parser for the given grammar.

lexer.y (YACC file)

```
%{
#include <stdio.h>
```

```
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
%}
%token ID
%%
expr : ID
     | expr '+' ID
     | expr '-' ID
     ;
%%
void yyerror(const char *s) {
   printf("Invalid string\n");
}
int main() {
   printf("Enter expression using id: ");
   if (yyparse() == 0)
      printf("Parsing successful\n");
   return 0;
}
```

2. lexer.l (LEX file)

```
%{
#include "y.tab.h"
%}
%%
[a-zA-Z]+  { return ID; }
"+"        { return '+'; }
"-"        { return '-'; }
[ \t]      ;
\n         { return 0; }
```

```
.       { return yytext[0]; }
%%
int yywrap() {
   return 1;
}
```

8. Write a YACC program to evaluate algebraic expressions.

lexer.l — LEX

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+"."[0-9]+  { return FLOAT; }
[0-9]+          { return INT; }
[+*/-]          { return yytext[0]; }
[ \t]           ;
\n              { return 0; }
.               { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

2. lexer.y — YACC

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token INT FLOAT
%%
expr : INT '+' INT     { printf("Type: int\n"); }
     | FLOAT '+' FLOAT { printf("Type: float\n"); }
     | INT '+' FLOAT   { printf("Type Error: int + float\n"); }
```

```
    | FLOAT '+' INT   { printf("Type Error: float + int\n"); }
    ;
%%
void yyerror(const char *s) {
   printf("Invalid Expression\n");
}
int main() {
   printf("Enter expression: ");
   yyparse();
   return 0;
}
```

10. Implement any one storage allocation strategy (heap, stack, static)

```
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;
void push() {
   int x;
   if(top == MAX-1)
      printf("Stack Overflow\n");
   else {
      printf("Enter element: ");
      scanf("%d", &x);
      stack[++top] = x;
      printf("Element inserted: %d\n", x);
   }
}
void pop() {
   if(top == -1)
      printf("Stack Underflow\n");
   else
      printf("Deleted element: %d\n", stack[top--]);
```

```
}
void display() {
   int i;
   if(top == -1)
      printf("Stack is empty\n");
   else {
      printf("Stack elements:\n");
      for(i=top; i>=0; i--)
         printf("%d\n", stack[i]);
   }
}
int main() {
   int ch;
   printf("Stack Storage Allocation\n");
   while(1) {
      printf("\n1.PUSH  2.POP  3.DISPLAY  4.EXIT\n");
      printf("Enter choice: ");
      scanf("%d", &ch);
      switch(ch) {
         case 1: push(); break;
         case 2: pop(); break;
         case 3: display(); break;
         case 4: return 0;
         default: printf("Invalid choice\n");
      }
   }
}
```

11 Write a recursive descent parser for the grammar

a. E->E+T

E->T

T->T*F

T->F

F->(E)/id.

```
#include <stdio.h>
char s[100];
int i=0;
void E(), T(), F();
void E() {
   T();
   while(s[i]=='+') {
      i++;
      T();
   }
}
void T() {
   F();
   while(s[i]=='*') {
      i++;
      F();
   }
}

void F() {
   if(s[i]=='i') i++;
   else if(s[i]=='(') {
      i++;
      E();
      if(s[i]==')') i++;
      else return;
   }
}
int main() {
```

```
    printf("Enter expression: ");
    scanf("%99s",s);
    E();
    if(s[i]=='\0') printf("Valid Expression");
    else printf("Invalid Expression");
    return 0;
}
```

b. S->(L)

S->a

L->L, S

L->S

```
#include <stdio.h>
char s[100];
int i=0;
int S() {
    if(s[i]=='a') i++;
    else if(s[i]=='(') {
        i++;
        if(!S()) return 0;
        while(s[i]==',') {
            i++;
            if(!S()) return 0;
        }
        if(s[i++ ]!=')') return 0;
    }
    else return 0;
    return 1;
}
int main() {
    printf("Enter string: ");
    scanf("%99s",s);
```

```
    if(S() && s[i]=='\0')
        printf("Valid String");
    else
        printf("Invalid String");
    return 0;
}
```

12 Write a C program to calculate the first function for the grammar

E->E+T

E->T

T->T*F

T->F

F->(E)/id

```
#include <stdio.h>
#include <string.h>
char p[10][20], first[10][20];
int n;
int main() {
    int i, j, k;
    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (without spaces):\n");
    for(i=0; i<n; i++)
        scanf("%s", p[i]);
    for(i=0; i<n; i++) {
        char ch = p[i][0];
        if(p[i][3] == '(')
            strcat(first[i], "(");
        else if(p[i][3] == 'i')
            strcat(first[i], "id");
        else {
```

```
        for(j=0; j<n; j++)
            if(p[j][0] == p[i][3]) {
                if(p[j][3] == '(')
                    strcat(first[i], "(");
                else if(p[j][3] == 'i')
                    strcat(first[i], "id");
            }
        }
    }
    printf("\nFIRST Sets:\n");
    for(i=0; i<n; i++)
        printf("FIRST(%c) = { %s }\n", p[i][0], first[i]);
    return 0;
}
```

Additional programs

1. LEX program to count vowels in a string

```
%{
#include<stdio.h>
int count = 0;
%}

%%
[aeiouAEIOU]  { count++; }
.           ;
\n          { return 0; }
%%
int main()
{
    printf("Enter a string: ");
    yylex();
    printf("Number of vowels = %d\n", count);
```

```
        return 0;
    }
    int yywrap()
    {
        return 1;
    }
```

Program 2: Count characters, tabs, spaces and reserved words using LEX/FLEX

```
%{
#include<stdio.h>
int ch=0, tab=0, space=0, key=0;
%}
%%
"int"|"float"|"char"|"if"|"else"|"while"|"return"  {key++;}
\t      {tab++;}
" "     {space++;}
.       {ch++;}
\n      ;
%%
int main()
{
   printf("Enter input:\n");
   yylex();
   printf("\nCharacters = %d",ch);
   printf("\nTabs = %d",tab);
   printf("\nSpaces = %d",space);
   printf("\nReserved words = %d",key);
   return 0;
}
int yywrap()
{
   return 1;
```

```
}
```

Program 3: LEX program to check whether an input integer is positive or negative

```
%{
#include<stdio.h>
%}
%%
[0-9]+   {
          if(atoi(yytext)>0)
             printf("Positive number");
          else
             printf("Zero");
         }
-[0-9]+   { printf("Negative number"); }
\n       ;
%%
int main()
{
   printf("Enter an integer: ");
   yylex();
   return 0;
}
```

Program 4: Pattern Matching

```
%{
#include<stdio.h>
int abc = 0, xyz = 0, pqr = 0;
%}
%%
abc      { abc++; }
xyz      { xyz++; }
pqr      { pqr++; }
\n       { return 0; }
```

```
.|\t    ;
%%
int yywrap()
{
   return 1;
}
int main()
{
   printf("Enter text:\n");
   yylex();
   if(abc > 0)
      printf("\nPattern abc found %d time%s\n", abc, abc==1?"":"s");
   if(xyz > 0)
      printf("Pattern xyz found %d time%s\n", xyz, xyz==1?"":"s");
   if(pqr > 0)
      printf("Pattern pqr found %d time%s\n", pqr, pqr==1?"":"s");
   return 0;
}
```

Program 5: Count and display words with length > 5 and < 10

```
%{
#include<stdio.h>
#include<string.h>
int count=0;
%}
%%
[a-zA-Z]{6,9} {
   count++;
   printf("%s - Length: %d\n",yytext,yyleng);
}
[a-zA-Z]+ ;
[ \t\n]+ ;
```

```
. ;
%%
int main()
{
   printf("Enter a string:\n");
   yylex();
   printf("\nTotal words = %d\n",count);
   return 0;
}
int yywrap()
{
   return 1;
}
#include<stdio.h>
#include<string.h>
#include<ctype.h>

char l[10],r[10][20],f[26][20];

void add(char *s,char c)
{
   if(!strchr(s,c))
   {
      int n=strlen(s);
      s[n]=c;
      s[n+1]='\0';
   }
}
```

6. Write a C program to calculate FOLLOW set function for the grammar.

```
int main()
{
```

```
int n,i,j,k,ch=1;
char s[50];
printf("Enter number of productions: ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
   scanf("%s",s);
   l[i]=s[0];
   strcpy(r[i],strchr(s,'>')+1);
}
add(f[l[0]-'A'],'$');
while(ch)
{
   ch=0;
   for(i=0;i<n;i++)
      for(j=0;r[i][j];j++)
         if(isupper(r[i][j]))
         {
            char *a=f[r[i][j]-'A'];
            if(r[i][j+1] && !isupper(r[i][j+1]))
            {
               if(!strchr(a,r[i][j+1]))
               {
                  add(a,r[i][j+1]);
                  ch=1;
               }
            }
            else if(!r[i][j+1])
            {
               char *b=f[l[i]-'A'];
               for(k=0;b[k];k++)
```

```
                    if(!strchr(a,b[k]))
                    {
                       add(a,b[k]);
                       ch=1;
                    }
                }
            }
    }
    for(i=0;i<n;i++)
    {
       for(j=0;j<i && l[j]!=l[i];j++);
       if(j<i) continue;
       printf("FOLLOW(%c)={%s}\n",l[i],f[l[i]-'A']);
    }
    return 0;}
```

7.Write a YACC program to convert an infix expression into a postfix expression.

lexer.l — LEX

```
%{
#include "y.tab.h"
#include <stdio.h>
%}
%%
[a-zA-Z0-9]+  { printf("%s ",yytext); return OPERAND; }
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
"("         { return '('; }
")"         { return ')'; }
[ \t]       ;
\n          { return 0; }
```

```
.           { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

2. lexer.y — YACC

```
%{
#include<stdio.h>
int yylex(void);
int yyerror(char *s);
%}
%token OPERAND
%left '+' '-'
%left '*' '/'
%%
S : E ;
E : E '+' E { printf("+ "); }
  | E '-' E { printf("- "); }
  | E '*' E { printf("* "); }
  | E '/' E { printf("/ "); }
  | '(' E ')'
  | OPERAND
  ;
%%
int main(){
   printf("Enter infix expression:\n");
   yyparse();
   printf("\n");
   return 0;
}
int yyerror(char *s){
   printf("\nInvalid expression\n");
   return 0;}
```